

Pushdown Is Not Free: A Small-Scale Measurement of Storage-Side PII Redaction in Text-Cleaning Pipelines

Feilian Huang *Independent Researcher*

Abstract

Running data cleaning on storage nodes (“pushdown”) promises less data transfer and keeps sensitive content inside storage. We measure this trade-off for an LLM text-cleaning pipeline (heuristic filtering, language identification, regex PII redaction, MinHash signing) over 20k FineWeb documents with synthetic PII at three densities. We compare pull-everything (A), full pushdown under storage CPU quotas of 10/25/50% (C), a same-region CPU cluster (E), and redaction-only pushdown (R), using a single-machine simulation with a bandwidth model. Full pushdown is slower than A in every configuration (n=3 per cell, 95% CIs): 1.96x at 50% quota, 3.49–3.56x at 25% quota, and 7.83x at 10% quota. The cause is structural: the workload is compute-bound (transfer is under 1% of A’s time at 100 Mbps) and filtering removes about 1% of bytes, so there is nothing for pushdown to save. A first-order cost model shows that, at 50% quota, no reduction ratio makes full pushdown win above roughly 0.88–1.15 Mbps. What pushdown does provide is a guarantee that regex-detectable PII is redacted before leaving storage. That guarantee costs 3.49–3.56x under full pushdown at 25% quota, but only 1.21–1.28x when just the redaction step is pushed (scheme R, single run per cell; 1.09–1.51x across quotas on d10).

1 Introduction

Modern LLM datasets are built by cleaning web-scale text with heuristic quality filters, language identification, PII scrubbing, and deduplication, as in the DataTrove/FineWeb pipeline [8]. In the standard architecture this work runs on a compute cluster after raw data has been pulled across the network, while storage-node CPUs sit largely idle.

Pushdown moves per-document cleaning steps onto storage nodes, so that only cleaned data crosses the network and sensitive content never leaves storage unredacted. Pushdown has been studied for adjacent problems: training-time image preprocessing on object stores [9, 14], computation offloading to disaggregated NVMe-oF storage [7], and query pushdown to S3 [18, 16]. It has not, to our knowledge, been measured for text-cleaning pipelines, whose workload shape differs: filter rates on pre-cleaned corpora are low, regex and language-identification steps are CPU-heavy per byte, and global deduplication cannot be pushed down at all.

This paper reports a small, single-machine measurement of that setting. At the project outset we wrote down four hypotheses (H1–H4, Section 4.4) in an internal plan; they were not publicly registered. After early results showed no performance advantage for full pushdown, we added a compliance analysis (Section 4.3). That analysis is exploratory and was not planned in advance.

Our contributions are:

1. A pushdown benchmark harness with declarative pipeline specs, pluggable storage backends, and a per-document output-hash correctness check across placement schemes.

2. A 38-run measurement: an 11-cell matrix (pull-everything, full pushdown at three storage CPU quotas, same-region CPU cluster) with $n=3$ repeats per cell, plus a 5-run redaction-only pushdown pilot, on a PII-redaction pipeline with three injected PII densities.
3. A first-order cost model (Section 3) that explains the measurements and gives a break-even condition for pushdown in terms of compute intensity, byte reduction, bandwidth, and quota.
4. A measured time premium for storage-side PII redaction under full pushdown, together with an explicit threat model and the limits of what that guarantee covers.

2 Design

2.1 Placement schemes

- **A (pull everything):** the compute cluster reads all raw documents from storage and runs the full pipeline locally. This is the status quo.
- **C (full pushdown):** every per-document step runs on the storage side and only its output crosses the network. Storage CPU is throttled to 10%, 25%, or 50% to model a quota, since production storage nodes must reserve cycles for foreground I/O.
- **E (same-region CPU cluster):** the pipeline runs on unthrottled CPUs in the same region as the data, modeled with a 10 Gbps link. This moves compute to the data without changing the storage software.
- **R (redaction-only pushdown):** only `pii_redact` runs on the storage side under the CPU quota; redacted documents are pulled and the remaining steps run on compute. This is the minimal placement that satisfies the compliance requirement in Section 4.3.

R redacts before `rule_filter` rather than after it (in A, filtering runs before redaction), so [REDACTED] tokens can change the uppercase/symbol ratios of borderline documents. The outputs therefore differ slightly: identical to A on d01, one document different on d10, ten documents different on d30. R is internally hash-consistent across its runs and passes the per-tier correctness gate; the difference is a disclosed filter-order effect, not a correctness failure.

A split scheme (D: MinHash signatures on storage, LSH bucketing on compute) and column pruning are not measured here (Section 7).

2.2 Pipeline

The pipeline is a FineWeb-style local cleaning pipeline:

1. `rule_filter`: Gopher/FineWeb-style heuristics (length, symbol ratio, uppercase ratio).
2. `language_id`: keep documents classified as English with confidence above 0.9.
3. `pii_redact`: regex redaction of email addresses, US phone numbers, and US SSNs, each replaced with [REDACTED] (verbatim patterns in Section 2.4).
4. `minhash_sign`: 128-permutation MinHash signatures (computed on storage in C, on compute in A).

5. `exact_dedup`: global exact deduplication, on compute in every scheme because it is cross-shard.

Signatures are computed in both schemes but consumed in neither: they are written to sidecar files in C (about 570 B per document compressed, roughly 11 MB per tier) that never cross the network, and in A they are computed and discarded. No step consumes them, because step 5 is exact rather than near-duplicate deduplication (sha256 of the text); the signatures exist for a future scheme-D-style LSH step that is not measured here. In a real FineWeb-style flow the signatures would be consumed, and a split scheme that computes them on storage but buckets them on compute would have to ship them across the network: at about 570 B per signature against about 1.25 KB per compressed document, the signatures are nearly half the document size, so moving them would erase most of the byte saving that pushdown is meant to provide. Their compute cost is included in both schemes' CPU time, and Table 1's byte counts correctly exclude them.

All runs use seed 42. Outputs are sorted by document id and hashed per document.

2.3 PII injection

We inject synthetic PII (no real personal data) into a shared base of 20k FineWeb documents at three densities: d01 (1% of documents, 208 instances), d10 (10%, 1,970 instances), and d30 (30% of documents, 1–3 per document, 12,001 instances). Before injection, we replaced 3,076 naturally occurring matches of the pipeline's own regexes with `[PRECLEARED]`, so that redaction counts reflect only injected instances. A build-time check runs `pii_redact` over each tier and confirms that the redaction count equals the injected count and that no regex match remains.

Because injected PII is generated to match the pipeline's regexes, this check verifies the plumbing, not the redaction method's recall on real PII (Section 4.3).

2.4 Experimental setup

Storage and compute run as co-located processes on one machine. All 38 runs executed on a 2-vCPU AMD EPYC 9D25 machine with 7.7 GiB RAM, no swap, Linux 7.0.0-38-generic (x86_64), Python 3.12.3. The pipeline is single-threaded per document with one worker process per input chunk (each run used one 20k-document chunk, so one process): A and E run unthrottled and use the full core. For C, the storage worker is a single `_worker.py` process per node (one node in every run), wrapped with `cpulimit -l N` without the `-i` flag, which is moot for a single-process worker; N is the percent of a single core (e.g., `-l 25` caps the worker at 25% of one core, i.e., quota $q = 0.25$). The harness falls back to a bundled 100 ms SIGSTOP/SIGCONT duty-cycle throttler (`scripts/throttle.py`) when `cpulimit` is unavailable; this happened once, for one of the three original d10 C-q25 repeats, which was later replaced by three clean `cpulimit` reruns (Section 4.1). Every run's JSON records which throttler it used.

Reproducibility details. Language identification uses the `langdetect` library with a fixed seed, scoring the first 2,000 characters and keeping English at confidence at least 0.9. The rule-filter thresholds are at least 50 words, at most 10% symbol characters, and at most 20% uppercase characters. The three redaction regexes are, verbatim: email `[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}`, US phone `(?<!\d)(?:\+1[-.\s])?(?:\(\d{3}\)|[-.\s])\d{3}[-.\s]\d{4}(?!\\d)`, and US SSN `(?<!\d)\d{3}-\d{2}-\d{4}(?!\\d)`. MinHash uses 128 permutations over 5-word shingles with seed 42. Input and all outputs are gzip-compressed JSONL. Run order was not randomized and the page cache was not controlled (warm after the first run touching a file).

Transfer time is computed from a bandwidth model: 100 Mbps for the cross-cloud path used by A and C, and 10 Gbps for E. The modeled transfer time is computed after each run as `bytes_served`

/ `bandwidth` and added to the measured wall times (`time_to_ready = storage_wall + transfer_time + compute_wall`); it is not injected during the run (e.g., via sleep). Storage work, transfer, and compute are treated as sequential, with no overlap. All schemes share the same model, so ratios between schemes are internally consistent, but absolute times do not transfer to real clouds.

The main matrix has 11 cells: A, C at 25% quota, and E once per tier (9 runs), plus C at 10% and 50% quota on d10 (2 runs). Each cell was repeated twice more ($n=3$ total) with 95% CIs from $t = 4.303$. Scheme R ran once per cell (5 runs: d01/d10/d30 at 25% quota, d10 at 10% and 50% quota), so R ratios have no error bars. 38 runs in all. Input is chunked gzip JSONL in all runs.

The correctness check groups runs by tier. Within each tier, A, C, and E produce identical per-document output hashes (R differs slightly, Section 2.1); across the matrix runs there are exactly three distinct hashes, one per tier. The released report script (`scripts/20_report.py`) groups the correctness gate by tier label for this reason: tiers differ in injected PII, so their hashes legitimately differ, and comparing across tiers was a false FAIL in an earlier report version. With tier grouping, all three tiers PASS, and the tier labels in Tables 1–2 match the hash grouping verified from the raw run files.

3 A First-Order Cost Model

Let B be input bytes, ρ the fraction of bytes that survive the per-document steps, β the cross-region bandwidth, W the CPU time of the per-document steps on an unthrottled core, G the time of the global steps on compute, and q the storage CPU quota. Ignoring overlap between transfer and compute:

$$\begin{aligned} T_A &\approx B/\beta + W + G \\ T_C &\approx \rho B/\beta + W/q + G \end{aligned}$$

Pushdown is faster only if the bytes it saves outweigh the compute it slows down:

$$(1 - \rho) \cdot B/\beta > W \cdot (1/q - 1) \quad (1)$$

Dividing by B gives the condition in per-byte terms, with $\kappa = W/B$ the compute intensity of the pushed steps:

$$(1 - \rho)/\beta > \kappa \cdot (1/q - 1) \quad (2)$$

In our setup q is a fraction of one core. In a deployment, the relevant quantity is the ratio of storage-side CPU capacity available to the pipeline to the compute-side capacity, and the model applies with q read that way. That reading has a regime our experiment does not cover: with many storage nodes, aggregate storage-side CPU can exceed compute-side CPU (for example, 100 nodes each contributing 10% against an 8-node compute cluster), i.e., $q > 1$. There the $W(1/q - 1)$ term is negative and pushdown wins on parallelism even with $\rho = 1$. Our measurements cover $q \leq 0.5$ and the paper’s claims cover $q \leq 1$; the $q > 1$ regime is a different trade-off that we do not measure.

For our workload, $B = 25$ MB is the gzip-compressed bytes of the input chunk files as stored on disk, i.e., the bytes that cross the network in the bandwidth model, so B/β is 2.0 s at 100 Mbps and 0.02 s at 10 Gbps, less than 1% of T_A . Two estimates of κ come from the d10 measurements. First, the measured difference is $T_C - T_A = W(1/q - 1) + O - (1 - \rho)B/\beta$, where $O \geq 0$ is any quota-independent overhead of C (Section 4.1) and the transfer term is at most 0.02 s here. At $q = 0.5$ the difference is $465.9 - 238.2 = 227.7$ s, so $W \leq 227.7$ s and $\kappa \leq 9.1$ s/MB. Second, the quota-scan fit in Table 2 gives $W \approx a = 174.2$ s, so $\kappa \approx 7.0$ s/MB. We use the range 7.0–9.1 s/MB.

Table 1: Time-to-ready by tier (mean $\pm$ 95% CI, $n=3$ per cell; R is single-run). Ratios are relative to A within the same tier.

tier	A (s)	E (s)	E/A	C, 25% (s)	C/A	R, 25% (s)	R/A
d01	229.8 ± 6.5	229.4 ± 6.4	1.00	807.3 ± 33.2	3.51	295.0	1.28
d10	238.2 ± 21.9	231.3 ± 4.3	0.97	832.4 ± 39.7	3.49	288.7	1.21
d30	230.3 ± 8.9	236.1 ± 24.1	1.03	819.4 ± 63.8	3.56	283.9	1.23

Table 2: Quota scan on d10 ($A = 238.2 \pm 21.9$ s). The last column is a least-squares fit $T_C = a/q + b$ over the three cell means, with $a = 174.2$ s and $b = 299.4$ s.

quota q	C (s)	C/A	$1/q$	fit (s)
10%	1864.9 ± 66.9	7.83	10	1867.4
25%	832.4 ± 39.7	3.49	4	822.1
50%	465.9 ± 8.4	1.96	2	473.6

At 100 Mbps, $1/\beta = 0.08$ s/MB. The left side of the condition is at most 0.08 s/MB even if pushdown discarded every byte ($\rho = 0$), while the right side is 7.0–9.1 s/MB at $q = 0.5$ and larger at lower quotas. Full pushdown therefore cannot win on this workload at 100 Mbps at any quota $q \leq 0.5$, regardless of byte reduction. This does not depend on the unconsumed MinHash step: removing its roughly 62% share of W (Section 4.3) leaves κ at about 2.6–3.5 s/MB, still two orders of magnitude above 0.08 s/MB. At $q = 0.5$, the break-even bandwidth is about 0.88–1.15 Mbps with total byte elimination, and about 8.8–11.5 kbps at the observed reduction of about 1%; at lower quotas it is lower still.

The model does not rule out quotas near 1. As $q \rightarrow 1$ the compute term vanishes, and the model predicts that any byte reduction makes pushdown win. What prevents that in our data is the overhead O , which we observe only by extrapolation (Section 4.1). The conclusion is therefore limited to $q \leq 0.5$.

The model also predicts which pushdown could pay: steps with low κ and large byte reduction, such as cheap heuristic filters applied to raw, unfiltered crawl data. This is consistent with the selective-pushdown argument of SOPHON.

4 Evaluation

4.1 Full pushdown is slower at every quota

Every run transferred about 0.025 GB (C/A byte ratios of 0.99–1.00 across tiers) because the filters remove only about 1% of documents and redaction does not shrink text.

Three observations follow.

First, the slowdown tracks $1/q$, as the model predicts for a compute-bound workload. The form $T_C = a/q + b$ fits the three cell means well: at $q = 50\%$ it gives 473.6 s against a measured 465.9 ± 8.4 s, and at $q = 10\%$ it gives 1867.4 s against 1864.9 ± 66.9 s. The slowdowns are below $1/q$ because part of the pipeline (global deduplication, I/O, parsing) is not throttled. The fit extrapolates to 299.4 s at $q = 1$, about 61 s above A. This suggests a quota-independent overhead O in C, but we treat the extrapolation as descriptive, not as a measured quantity. That overhead is not compute: C’s measured CPU time exceeds A’s by only about 15–25 s, so most of the extrapolated 61 s must come from the throttling mechanism itself (SIGSTOP/SIGCONT scheduling granularity

and I/O wait while stopped), not from the pipeline. One of the three original d10 C-q25 repeats ran notably slow (1067.7 s vs 841.3/829.5 s); it was the run that fell back to `throttle.py`'s 100 ms duty cycle instead of `cpulimit`, with matching storage CPU time (252.8 s vs 242.2/241.8 s), so the extra roughly 226 s of wall time is throttling overhead. That cell was re-run three times with `cpulimit` (842.1/841.2/814.0 s); the clean re-runs replace the original cell, and the fallback-throttler run is retained in the release as a throttler comparison.

Second, E and A are indistinguishable in time. The model predicts a 2 s difference (2.0 s vs 0.02 s of transfer), and E/A ranges from 0.97 to 1.03 across tiers, with the 95% CIs overlapping A's in every tier. E is a sanity check rather than an independent finding: on a single machine it differs from A only in the bandwidth model, and transfer is under 1% of total time, so $E \approx A$ is constructed. Run-to-run noise is directly quantified by the CIs (Section 5).

Third, the low byte reduction is a property of the input. FineWeb documents have already passed FineWeb's filters, so rerunning similar filters removes little. On raw crawl data the same filters would remove far more. By the model in Section 3, this alone would not rescue full pushdown at the measured κ . However, κ would itself change on raw data: filters that drop many documents early leave less text for the expensive later steps, lowering κ per input byte. The raw-data case therefore needs its own measurement.

4.2 Resource cost

We do not report dollar figures, because the harness uses placeholder prices. In resource terms, all three schemes move 0.025 GB: A and C across regions, E within a region. CPU time is 0.062–0.064 CPU-h for A and 0.062–0.065 CPU-h for E, versus 0.068–0.069 CPU-h for C at 10% quota, 0.066–0.071 CPU-h at 25% quota, and 0.067–0.068 CPU-h at 50% quota. R consumes 0.065–0.067 CPU-h at 25% quota, between A and C. C's CPU consumption is quota-independent (throttling stretches wall time, not CPU time) and consistently above A's. Two orderings follow without any particular price: E costs no more than A whenever intra-region transfer is priced at or below cross-region transfer, and C costs at least as much as A because it moves the same bytes and consumes at least as many CPU-hours. This assumes storage-side CPU is priced like compute-side CPU. The introduction motivates pushdown with idle storage CPUs; if their marginal cost is near zero, the $C \geq A$ resource ordering does not follow, and the time premium in Section 4.1 is the relevant cost. Our cost accounting charges only consumed CPU and does not charge compute nodes that sit idle while waiting on throttled storage, so it understates C's cost.

4.3 The compliance premium (exploratory)

Threat model. The storage tier lies inside a regulated trust domain. Anything outside the storage software, including the compute cluster reached over the 100 Mbps cross-cloud link, is outside it. The requirement is that raw documents never cross that boundary before redaction. Under this model, A and E both fail, because each pulls raw documents before redacting. Whether E fails in practice depends on whether same-region compute is inside the trust domain; if it is, E satisfies the requirement at no time premium.

Metric. We count detectable synthetic PII instances that leave storage unredacted. For A and E this equals the injected count by construction (208, 1,970, and 12,001). For C it is 0: the build-time check in Section 2.3 confirms this, and we additionally scanned C's transferred output with the pipeline's three regexes (email, US phone, US SSN) and found 0 matches (verified on the d10 q25

Table 3: Redaction-only (R) vs. full (C) pushdown, quota scan on d10 ($A = 238.2 \pm 21.9$ s). R is single-run; C is mean $\pm$ 95% CI, $n=3$.

quota q	R (s)	R/A	C (s)	C/A
10%	360.5	1.51	1864.9 ± 66.9	7.83
25%	288.7	1.21	832.4 ± 39.7	3.49
50%	259.0	1.09	465.9 ± 8.4	1.96

run: 19,792 transferred documents, 0 matches). These counts follow from the pipeline structure and are not independent measurements; the measured quantity is the time premium.

Premium. Keeping detectable PII inside storage under full pushdown costs 3.51x, 3.49x, and 3.56x time-to-ready at 25% quota for d01, d10, and d30, and 1.96x and 7.83x at 50% and 10% quota on d10 ($n=3$ means; d10 q25: 832.4 ± 39.7 s). The tier CIs overlap, so we draw no conclusion about how the premium varies with PII density.

Measured redaction-only premium. Scheme R pushes only `pii_redact`. In this harness the storage-side redaction and the compute-side steps run sequentially (redaction completes before compute pulls); they are not pipelined. R’s storage side does more than regex matching: the throttled worker also gunzips the input chunk, parses JSON lines, and re-compresses the redacted output, all under the quota. This explains a small mismatch: fitting $R-A$ against $(1/q - 1)$ gives a slope of about 12.5 s, roughly 7% of the fitted W , while per-stage profiling attributes only about 4% to `pii_redact` itself; the remainder is the throttled decompression, parsing, and serialization around it. R costs 1.28x, 1.21x, and 1.23x at 25% quota for d01, d10, and d30 (Table 1), and 1.51x, 1.21x, and 1.09x at 10%, 25%, and 50% quota on d10 (Table 3; single run per cell, no CI).

Full pushdown is 2.7–2.9x slower than R at 25% quota. At matched quota, the compliance guarantee itself costs +21–28% (+9–51% across the d10 quota scan). The rest of C’s premium is the price of pushing steps that compliance does not require. Part of it comes from `minhash_sign`, which C pushes but nothing in this pipeline consumes (Section 2.2): per-stage profiling on a 2,000-document d10 sample shows `minhash_sign` accounts for about 62% of the per-document CPU time (`language_id` about 33%, `pii_redact` about 4%, `rule_filter` about 1%), so most of what full pushdown moves to storage is work the compliance requirement never asks for. R’s exposure count is 0 in all cells, since every document is redacted before leaving storage regardless of the filter-order effect in Section 2.1.

What the guarantee does not cover. The guarantee is only as strong as the redactor. Regex redaction does not detect names, postal addresses, non-US phone formats, or free-text identifiers, and its recall on real PII is below 100%. On real data, scheme C would leak whatever the redactor misses. Encrypted transfer with redaction inside a trusted execution environment is an alternative that meets the same requirement; we do not evaluate it.

4.4 Hypotheses

- **H1 (below some quota, pushdown is slower than pull-everything):** supported for every quota tested ($q \leq 0.5$). The model in Section 3 explains why no quota in that range rescues full pushdown on this workload. Whether a quota near 1 would depends on the overhead O , which we observe only by extrapolation.

- **H2 (E wins single-round, pushdown wins over iterations):** the first clause is supported in the weak sense that E is never slower than A beyond noise. The iteration clause was not tested.
- **H3 (chunked compression changes the winner):** not tested; all runs use one format.
- **H4 (pushdown interferes with foreground reads above some quota):** not tested.

5 Limitations

1. **Single-machine simulation.** Transfer time is modeled, not measured. Ratios between schemes are internally consistent; absolute times do not transfer to real deployments.
2. **Repeated matrix, single-run R.** The 11 A/C/E cells have $n=3$ with 95% CIs; the effects (2–8x) are far larger than the noise, but differences between PII tiers are not. R ran once per cell, so its 1.09–1.51x premium has no error bars. The d10 C-q25 cell was re-run three times with `cpulimit` after one original repeat used the fallback `throttle.py` throttler (1067.7 s vs 841.3/829.5 s); the fallback run is retained in the release as a throttler comparison but excluded from the cell statistics, which use only the three homogeneous `cpulimit` runs.
3. **Small, pre-cleaned input.** 20k documents (25 MB) from FineWeb, which has already passed similar filters. This fixes ρ near 1 and limits generality. Section 3 gives the condition under which other inputs would change the result.
4. **Synthetic PII matched to the redactor.** Injected PII is built to match the regexes, so exposure counts are structural, and the redactor’s real-world recall is not measured.
5. **Resource-only costs.** No real cloud prices are used, and idle compute during throttled storage work is not charged.
6. **Unconsumed MinHash step.** `minhash_sign` is computed but not consumed (Section 2.2). It is included in W for A and C alike, but it inflates the cost of what full pushdown moves to storage, and so part of the C-R gap.
7. **Throttling model.** `cpulimit` approximates a storage CPU quota with SIGSTOP/SIGCONT and may not match the behavior of production storage schedulers.

6 Related Work

Near-data processing. Pushing computation to storage dates back to active disks [1, 10] and today continues in computational storage devices and standards [11]. OffloadFS [7] offloads RocksDB flush/compaction and image preprocessing to NVMe-oF storage nodes, reporting up to 3.36x and 1.85x improvements over OCFS2.

Training-time preprocessing. HAPI [9] splits transfer-learning DNNs so feature extraction runs on cloud object stores. SOPHON [14] shows that full pushdown of image preprocessing can hurt and selectively offloads per sample, reducing data traffic and training time by 1.2–2.2x. `tf.data` service [2], FastFlow [13], and Pecan [4] optimize input pipelines through disaggregation, offloading, and placement. These target image and audio preprocessing; text cleaning differs in its low byte reduction on pre-cleaned corpora and high per-byte CPU cost.

Database pushdown. PushdownDB [18] used S3 Select for filter and projection pushdown, reporting 6.7x speedup and 30% cost reduction on TPC-H. FlexPushdownDB [16] combines pushdown with caching; its journal extension [17] adds adaptive pushback under resource pressure, the same failure mode our quota scan shows for text cleaning.

Text-cleaning pipelines and PII. DataTrove/FineWeb [8], Dolma [12], and RedPajama [15] define large-scale text-cleaning pipelines that run entirely on compute clusters. Lee et al. [5] establish the value of deduplication, which we treat as the canonical unpushable step. Production PII detection typically combines patterns with NER models, as in Microsoft Presidio [6]; our regex redactor is a lower bound on realistic redaction cost. Confidential computing [3] offers an alternative route to compliance.

7 Conclusion

On a compute-bound text-cleaning workload with low byte reduction, full storage-side pushdown was 2.0–7.8x slower than pulling data to compute ($n=3$ per cell), and a first-order model shows that no reduction ratio would change that at 100 Mbps for any quota at or below 50%. Quota is the dominant variable, and the per-byte condition $(1 - \rho)/\beta > \kappa(1/q - 1)$ indicates where pushdown can pay: cheap, highly selective steps on raw data over slow links. The remaining case for storage-side cleaning is compliance, where full pushdown buys a guarantee on regex-detectable PII at a measured premium of 3.5–3.6x at 25% quota, but pushing only the redaction step costs just 1.2–1.3x at the same quota (1.1–1.5x across quotas). The compliance guarantee itself is cheap; the rest of full pushdown’s premium is the price of pushing steps that compliance does not require.

The most useful next steps are raw crawl input, a bandwidth sweep on real links, the split scheme D (MinHash signatures on storage, LSH bucketing on compute), per-step κ estimates from the per-stage profiling in Section 4.3, repeated R runs with confidence intervals, and the untested hypotheses H2–H4.

Artifact Availability

Code, pipeline specs, experiment scripts, and raw results are available at [TODO: repository URL] under the Apache 2.0 license. `scripts/run_pii_matrix.sh` reruns the 11-run matrix, `scripts/run_pii_extras.sh` adds the $n=3$ repeats and the redaction-only pilot, and per-document output hashes provide the cross-scheme correctness check.

References

- [1] Acharya, A., Uysal, M., and Saltz, J. H. Active Disks: Programming Model, Algorithms and Evaluation. ASPLOS 1998, 81–91.
- [2] Audibert, A., Chen, Y., Graur, D., Klimovic, A., Šimša, J., and Thekkath, C. A. tf.data service: A Case for Disaggregating ML Input Data Processing. SoCC 2023.
- [3] Costan, V., and Devadas, S. Intel SGX Explained. Cryptology ePrint Archive, Report 2016/086, 2016.
- [4] Graur, D., Mraz, O., Li, M., Pourghannad, S., Thekkath, C. A., and Klimovic, A. Pecan: Cost-Efficient ML Data Preprocessing with Automatic Transformation Ordering and Hybrid Placement. USENIX ATC 2024, 649–665.

- [5] Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., and Carlini, N. Deduplicating Training Data Makes Language Models Better. *ACL 2022*, 8424–8445.
- [6] Microsoft. Presidio: Data Protection and De-identification SDK. <https://github.com/microsoft/presidio>, 2024.
- [7] Moon, S., Han, D., Koo, H., Chae, S., Bae, D.-H., Seo, E., and Nam, B. OffloadFS: Leveraging Disaggregated Storage for Computation Offloading. *arXiv:2604.13743*, 2026.
- [8] Penedo, G., Kydlíček, H., Ben Allal, L., Lozhkov, A., Mitchell, M., Raffel, C., von Werra, L., and Wolf, T. FineWeb: Decanting the Web for the Finest Text Data at Scale. *arXiv:2406.17557*, 2024.
- [9] Petrescu, D., Guirguis, A., Le Quoc, D., Picorel, J., Guerraoui, R., and Dinu, F. Accelerating Transfer Learning with Near-Data Computation on Cloud Object Stores. *SoCC 2024*.
- [10] Riedel, E., Gibson, G. A., and Faloutsos, C. Active Storage for Large-Scale Data Mining and Multimedia. *VLDB 1998*, 62–73.
- [11] SNIA. Computational Storage Architecture and Programming Model, Version 1.0, 2022.
- [12] Soldaini, L., et al. Dolma: an Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. *ACL 2024*, 15725–15788.
- [13] Um, T., Oh, B., Seo, B., Kweun, M., Kim, G., and Lee, W.-Y. FastFlow: Accelerating Deep Learning Model Training with Smart Offloading of Input Data Pipeline. *PVLDB 16(5)*, 2023, 1086–1099.
- [14] Wang, M., Waldspurger, G., and Sundararaman, S. SOPHON: A Selective Preprocessing Offloading Framework for Reducing Data Traffic in DL Training. *HotStorage 2024*, 63–70.
- [15] Weber, M., et al. RedPajama: an Open Dataset for Training Large Language Models. *NeurIPS 2024 Datasets and Benchmarks Track*, 116462–116492.
- [16] Yang, Y., Youill, M., Woicik, M. E., Liu, Y., Yu, X., Serafini, M., Abounaga, A., and Stonebraker, M. FlexPushdownDB: Efficient Computation Pushdown for Cloud OLAP DBMSs. *PVLDB 14(11)*, 2021, 2101–2113.
- [17] Yang, Y., Yu, X., Serafini, M., Abounaga, A., and Stonebraker, M. FlexpushdownDB: Re-thinking Computation Pushdown for Cloud OLAP DBMSs. *The VLDB Journal 33(5)*, 2024, 1643–1670.
- [18] Yu, X., Youill, M., Woicik, M. E., Ghanem, A., Serafini, M., Abounaga, A., and Stonebraker, M. PushdownDB: Accelerating a DBMS Using S3 Computation. *ICDE 2020*, 1802–1805.